Available online at www.sciencedirect.com
SCIENCE DIRECT®
Computers & Industrial Engineering 46 (2004) 1–15
computers & industrial engineering
www.elsevier.com/locate/dsw

Dynamic rescheduling that simultaneously considers efficiency and stability

Ruedee Rangsaritratsamee[a], William G. Ferrell Jr.[b,*], Mary Beth Kurz[b]

[a]*Department of Instrumentation Engineering, King Mongkut's Institute of Technology, Ladkrabang (KMITL), Chalongkrung Road, Ladkrabang, Bangkok 10520, Thailand*
[b]*Department of Industrial Engineering, Clemson University, Box 340920, Clemson, SC 29634-0920, USA*

Accepted 11 September 2003

Abstract

Dynamic job shop scheduling is a frequently occurring and highly relevant problem in practice. Previous research suggests that periodic rescheduling improves classical measures of efficiency; however, this strategy has the undesirable effect of compromising stability and this lack of stability can render even the most efficient rescheduling strategy useless on the shop floor. In this research, a rescheduling methodology is proposed that uses a multiobjective performance measures that contain both efficiency and stability measures. Schedules are generated at each rescheduling point using a genetic local search algorithm that allows efficiency and stability to be balanced in a way that is appropriate for each situation. The methodology is tested on a simulated job shop to determine the impact of the key parameters on the performance measures.

Keywords: Dynamic scheduling; Genetic local search

1. Introduction

Job shops are production systems arranged to produce a number of different part types so that each type is allowed to have unique routing and a different processing time on each machine visited. The classic scheduling problem in the job shop assumes that a known collection of jobs is to be scheduled and the task is to arrange them in a sequence that optimizes a performance measure. This general problem is well known to be NP-hard, so algorithms and heuristics are required for all but very special situations. In many real systems, this scheduling problem is even more difficult because jobs

* Corresponding author. Fax: +1-864-656-0795.
E-mail address: fwillia@ces.clemson.edu (W.G. Ferrell).

0360-8352/$ - see front matter © 2003 Elsevier Ltd. All rights reserved.
doi:10.1016/j.cie.2003.09.007

arrive on a continuous basis, a problem henceforth called dynamic job shop scheduling (DJSS). Previous research on DJSS using classic performance measures like makespan or tardiness concludes that it is highly desirable to construct a new schedule frequently so recently arrived jobs can be integrated into the schedule soon after they arrive. The problem with this strategy is that constantly changing the production schedule can induce instability, a very undesirable effect in shop floor control and one that has been labeled 'nervousness' in some circles. In this research, a methodology to address DJSS is presented based on a bicriteria objective function that simultaneously considers efficiency and stability so the decision maker can strike a compromise between improved efficiency and stability.

Much of the research on DJSS uses traditional performance measures of efficiency like makespan and tardiness. The strategy that optimizes this problem structure typically involves frequently building new schedules so that arriving jobs can be integrated into the schedule as quickly as possible. This paper henceforth uses the term 'rescheduling' for this approach. The important point is that while rescheduling will optimize the efficiency measure, the strategy generates schedules that are often radically different from the previous one. This means that many, if not all, of the previously scheduled jobs that have not begun processing can have their start time accelerated or delayed. This effect is troublesome in practice, especially in the common situation where the process being scheduled uses material that must be delivered from external sources. For example, in an assembly operation, the material planner would be required to constantly expedite orders and potentially hold excess inventory for a long period of time trying to support each new schedule. In practice, this is simply unacceptable. Clearly, improving efficiency is important in systems that have dynamic job arrivals but the instability problem induced by unrestricted rescheduling renders the approach useless. The goal of this research is to improve schedule efficiency and maintain stability through a methodology that uses a local search genetic algorithm and a multiobjective performance measure.

While measures of efficiency are well known and have appeared in scheduling research for decades, only a few researchers have addressed the impact of disruptions induced by moving jobs during a rescheduling event. The impact is frequently called 'stability'; however, no universal definition exists for this term. Church and Uzsoy (1992) used the number of times rescheduling takes place as a measure of stability and suggested that more frequent rescheduling means a less stable schedule. Wu, Storer, and Chang (1993) defined stability in terms of the deviation of job starting times between the original and revised schedule is and the difference of job sequences between the original and revised schedules. One of the shortcomings of these approaches is that they ignore the fact that the impact of changes increases as they are made closer to the current period (Lin, Krajewski, Leong, & Benton, 1994). In this methodology, two dimensions of stability are modeled. The first captures the deviation of job starting times between two successive schedules and the second reflects how close to the current time changes are made.

The methodology is, then, multiobjective because both efficiency and stability are simultaneously addressed. Rescheduling is assumed to take place on a periodic basis and a local search genetic algorithm is used to obtain a good schedule relative to the multiple objectives that are combined into a single fitness function. A realistic example is then presented in this paper to serve two purposes. First, it illustrates how the methodology can be implemented. Second, and more importantly, it is used as a test platform to experimentally investigate the impact of some of the key parameters on the objectives.

2. The proposed methodology

2.1. Prototype process

There are many examples of DJSS ranging from large companies in the supply chain that produce automobiles to small shops making specialized replacement parts for textile machines. In this research, we consider a prototype system that possesses some of the most important features relative to the DJSS problem. The prototype system consists of M machines or single function workstations arranged in series with each machine performing a different operation. Jobs arrive dynamically through time so we denote that job n arrives at time a_n with due date DD_n. Since it is permissible for each job to be unique, the processing time required for job n on machine m is denoted $p_{n,m}$. Note that if $p_{n,m} = 0$, job n does not have to be processed on machine m. The scheduling strategy used in the prototype system is periodic rescheduling in which new schedules are constructed at specified time intervals using all jobs that are available at that moment. The objective is to construct schedules that effectively accommodate these dynamic arrivals by simultaneously maximizing efficiency and stability.

2.2. Strategy

Define the time at which a new schedule is constructed as the 'rescheduling point' and the time between two consecutive rescheduling points as the 'scheduling interval.' At each rescheduling point, all jobs from the previous schedule that have not begun processing are combined with jobs that arrived since the previous rescheduling point and a new schedule is built. The dynamics of the prototype problem have been constructed to preserve realism as closely as possible and make the problem manageable for analysis. In particular, it is assumed that a production horizon exists in which N total jobs are to be processed. An initial schedule is built at the beginning of the horizon with jobs that can be thought of as having arrived since the last rescheduling point of the previous planning horizon. The system is rescheduled at the rescheduling points and jobs are processed according to these schedules until all N jobs are complete and then the process repeats. Finally, as noted above, the details of a job are not known until the job arrives so it is not possible to look ahead with the schedules in anticipation of jobs that have yet to arrive.

2.3. Objective function

The methodology proposed in this research utilizes a multiobjective performance measure as the objective function to construct schedules. Specifically, the objective function consists of efficiency, as measured by makespan and tardiness, and stability, as measured by starting time deviation and a penalty proportional to the total deviation. Now, makespan traditionally is defined as the total time that is required to process a group of jobs. To fit the dynamic scheduling environment, this definition is modified so that the group of jobs includes all jobs scheduled at a scheduling point. Hence, makespan is calculated by simply taking the difference between the start time of the first job in a group of jobs that are scheduled and the completion time of the last job in that same group. Note that some jobs may be scheduled multiple times because they do not begin processing before the next rescheduling point is encountered. In these cases, the jobs are included in the makespan calculation at each rescheduling point. Also, there is a makespan measure associated with the schedule created at each rescheduling point.

Tardiness is defined using the traditional approach, namely, the difference between the completion time and due date for each job in which the completion time occurs after the due date. Ishibuchi and Murata (1998) observed that 'the variance of the makespan is much smaller than that of tardiness' and noted that this can have a detrimental effect on the quality of solutions as well as convergence time when genetic algorithms are employed to resolve multiobjective problems using the weighting method as we propose. They recommended that constant multipliers of 5 and 2 be assigned to makespan and tardiness, respectively (Ishibuchi & Murata, 1998). Hence, the quantity to be minimized is:

$$\text{Efficiency} = 5 \times \text{Makespan} + 2 \times \text{Tardiness} = 5[\underset{\text{all } n}{\text{Max}}\,(d_n) - \underset{\text{all } n}{\text{Min}}\,(s_n)] + 2\sum_{\text{all } n} \psi_n (d_n - \text{DD}_n) \tag{1}$$

where

d_n, departure time of job n
DD_n, due-date of job n
s_n, starting time of job n
$\psi_n = 1$ if $(d_n - \text{DD}_n) > 0$; $= 0$ otherwise.

Stability is also measured by two components, both of which are based solely on the status of jobs that were scheduled at one scheduling point but do not enter processing before the next rescheduling point. The first measure is the total deviation for all such jobs between the starting times in a new schedule and old schedule (Wu et al., 1993). As mentioned before, this measure is important because of the impact changes have on external resources associated with processing jobs. The second measure associates a penalty with rescheduling jobs to earlier times (Lin et al., 1994). While it is true that rescheduling jobs later also has an impact, this research focuses solely on moving jobs to earlier times. One possible representation of this concept is presented in Fig. 1 where $\text{PF}(\gamma)$ is the penalty associated with total deviation from the current time, γ. For example, assume that job n begins its routing on machine m and was scheduled at $t_{n,m}$ but did not enter processing when the next rescheduling point was reached. Job n is then rescheduled on machine m at $t'_{n,m}$. Recall that this is possible because it has been assumed that all jobs available at each rescheduling point are included in the new schedule. If the current time is t, we define the total deviation as $\gamma = (t_{n,m} - t) + (t'_{n,m} - t) = t_{n,m} + t'_{n,m} - 2t$. Since $t_{n,m} > t$ and $t'_{n,m} > t$,

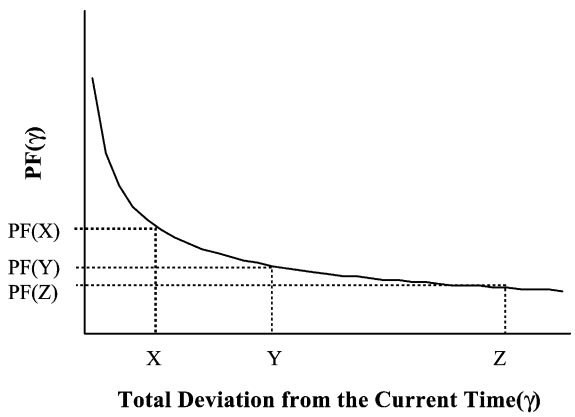

Fig. 1. A typical penalty function associated with total deviation from the current time.

the total deviation is always positive. If a job holds the same position in two successive schedules, then $t_{n,m} = t'_{n,m}$ and the total deviation is $\gamma = 2t_{n,m} - 2t$. The fact that this is not zero is of no consequence in this work because the objective function is multiobjective and the objectives have been scaled; hence, the value of the objective function has no physical meaning and must only be an internally consistent measure of how different schedules perform. As an example, consider Fig. 1 where a deviation has been determined to be Z and the associated penalty is PF(Z). If the job is rescheduled to begin processing at a time sooner than $t_{n,m}$, then $\gamma = (t_{n,m} + t'_{n,m}) - 2t < 2t_{n,m} - 2t$ because $t'_{n,m} < t_{n,m}$. This deviation is illustrated as point Y in the figure and note that PF(Y) > PF(Z). Similarly, if the rescheduled time is even earlier, X and PF(X) would result. Finally, it should not be assumed that Fig. 1 is the only representation of the phenomenon described by Lin et al. (1994); rather, it is one way to include this measure of stability in a quantitative model. In particular, we use $F(\gamma) = 10/(\gamma^{1/2})$ to generally match the shape and, to maximize stability, this penalty function is added to the starting time deviation and the sum is minimized. Hence, the following penalty is computed for all eligible jobs, that is, jobs that were previously scheduled but did not enter processing when the next rescheduling point was reached.

$$\text{Stability} = \text{Starting time deviation} + \text{Total deviation penalty}$$

$$= \sum_{\text{all eligible } n} \sum_{m} |t'_{n,m} - t_{n,m}| + \sum_{\text{all eligible } n} \sum_{m} \text{PF}(t'_{n,m} + t_{n,m} - 2t) \tag{2}$$

where

$t_{n,m}$, starting time of job n on the machine m per a schedule built at one rescheduling point
$t'_{n,m}$, starting time of job n on the machine m per a schedule built at the next rescheduling point
t, current time
PF(γ), penalty function associated with total deviation from the current time.

The proposed methodology combines Eqs. (1) and (2) to form the objective function represented by Eq. (3) that is used to build schedules at each rescheduling point that will possess a compromise between efficiency and stability.

$$\text{Min } z = \text{Efficiency} + \text{Stability}$$

$$= \left\{ 5\left[\operatorname{Max}_n (d_n) - \operatorname{Min}_n (s_n) \right] + 2 \sum_n \psi_n (d_n - \text{DD}_n) \right\}$$

$$+ \left\{ \sum_m \sum_n |t'_{n,m} - t_{n,m}| + \sum_m \sum_n \text{PF}(t'_{n,m} + t_{n,m} - 2t) \right\} \tag{3}$$

2.4. Additional requirements

To implement the methodology in the simulated environment, several additional requirements are imposed to determine a schedule. Some of these simply insure that the simulation reflects reality like

requiring that job j must be completed before job n can begin if job j immediately precedes job n on machine m. A unique requirement related to scheduling a machine when two jobs arrive at the same time. Specifically, it is assumed that the job released to the system earliest in the original schedule of the planning horizon is given a higher priority and released first to the machine. Since jobs are allowed to have unique routings and different processing times on each machine in the simulation, it is easy to visualize two simulated jobs arriving to the same machine at the same time. In these cases, the one that was started first in the original schedule is given priority. Finally, Eq. (1) is used as the objective function to build the initial schedule because a prior schedule does not exist; hence, there is no stability concern.

2.5. Genetic local search scheduler

Genetic algorithms have been successfully used for several years to find good solutions to a variety of single objective job shop scheduling problems. Recently, several researchers have suggested using a hybrid genetic algorithm consisting of a pure genetic algorithm with another search procedure (Bierwirth, Kopfer, Mattfeld, & Rixen, 1995; Murata, Ishibuchi, & Tanaka, 1996a). The most common forms of hybrid genetic algorithms are genetic local search and genetic simulated annealing. Murata et al. (1996a) concluded that the genetic local search outperforms the genetic simulated annealing in a comparison test. We note that genetic local search has proven successful in other scheduling settings as well (Ishibuchi & Murata, 1998; Yamada, 1998).

Genetic local search is used in this research to find an approximate solution to the multiobjective problem and the associated requirements. In this application, genetic operators are first used to perform a global search among the population and then a local search technique is applied to the promising areas identified by the main operation. The performance of each schedule is indicated by the value of the multiobjective function with the best schedule defined as the one with the smallest value. Now, fitness in genetic algorithms typically means greater is better, so the actually implementation uses a simple transformation of the multiobjective function as described in Goldberg (1989). A description of the genetic local search procedure is provided in Appendix A.

3. Example

To illustrate the methodology and to investigate the impact of including the stability terms in the objective function, a realistic job shop has been simulated. The nature and scope of this simulation has been synthesized from the work of others including Baker (1984), Bertrand (1983), Blocher, Chhajed, and Leung (1998), Chang (1996), Morton and Pentico (1993), Ragatz and Mabert (1988), and Rohleder and Scudder (1993). Most of these studies contain between four and 10 machines, so six machines are selected meaning each job will visit between one and six machines in its sequence. The number of machines is assigned randomly according to a uniform distribution. A purely random sequence is utilized to represent the more difficult control problem (Ragatz & Mabert, 1988); consequently, each job has its own sequence with possibly unique processing times at each machine. Finally, the exponential distribution with a mean of 1.0 time units is used to represent the processing times (Blocher et al., 1998; Chang, 1996; Pegden, Shannon, & Sadowoski, 1995).

3.1. Arrival time assignment

The average time between arrivals is established using machine utilization and individual arrival times, and are determined based on a distribution with this mean. With the average processing time of the machine established, the average arrival rate of jobs must be selected such that the utilization is less than 100% or the number of jobs in the queue will grow without bound. In this simulation, 80% shop utilization is employed. Many authors have concluded from investigations of empirical data from job shops and related operations that the distribution of the arrival process closely follows the Poisson distribution (Karsiti, Curz, & Mulligan, 1992; Pegden et al., 1995); hence, the time between arrivals of jobs is exponentially distributed. Here, the interarrival times for jobs are generated from the exponential distribution with mean calculated from Eq. (4) (Blocher et al., 1998; Chang, 1996; Karsiti et al., 1992):

$$b = \frac{1}{\lambda} = \frac{\mu_p \mu_g}{Um} \tag{4}$$

where

b, interarrival time
U, shop utilization
λ, mean job arrival rate
μ_p, mean processing time per operation
μ_g, mean number of operations per job
m, number of machines in the shop.

In the simulated prototype system used in this numerical example, it is assumed that $U = 0.8$, $\mu_p = 1.0$, $\mu_g = 3.5$, $m = 6$; hence, $b = 0.73$.

3.2. Due date assignment

Many classes of due date patterns have been studied and some simple due date assignment heuristics have been developed based on information of job characteristics that match these real patterns (Baker, 1984; Baker & Bertrand, 1981; Cheng, 1988). The total work content (TWK) rule has been found superior in many cases and is adopted in this work (Baker, 1984; Chang, 1996; Kanet & Christy, 1989). Using the TWK rule, Blocher et al. (1998) suggest that the due date of each job equals the sum of the job arrival time and a multiple of the total job processing time. The multiple is related to job characteristics and is called the tightness factor. Hence, the due date of job n is computed as:

$$\mathrm{DD}_n = a_n + K \sum_m p_{n,m} \tag{5}$$

where K is the tightness factor. This tightness factor is assigned from a normal distribution with a mean of 10 according to the recommendation of Blocher et al. (1998). Further, since it is recommended that the minimum value of the tightness factor be 2, the standard deviation of the distribution is set to 2 so meeting this minimum value is assured with near certainty.

3.3. Genetic algorithm parameters

The genetic algorithm used to build schedules is rather conventional and requires several parameters to be established. The settings used in this example have been shown to be effective in job shop scheduling by Ishibuchi and Murata (1998) and Murata et al. (1996a,b).

- Population size (Z) equals 10
- Crossover probability is 1.0
- Mutation probability is 0.1
- Number of neighborhood solutions examined in each local search procedure is 2
- Stopping condition is satisfied after 10,000 schedules are evaluated.

3.4. Measuring steady state performance

A Monte Carlo simulation of the job shop and the dynamic scheduling procedure were developed in Visual Basic. The simulation was verified and validated to insure that it performs as intended and is an accurate representation of the system under this study. It was experimentally determined that information from the first 230 arrivals should be eliminated from computations to remove transient effects. Hence, each simulation run in this study consists of 1230 arrivals of which the final 1000 are used to compute the performance measurements reported.

3.5. Experimental design

A full factorial design is used to experimentally investigate the impact that the stability term in objective function and the length of the scheduling interval have on the performance of the methodology as measured by stability and efficiency. Hence, experiments are conducted with two types of objective functions, the multiobjective approach using Eq. (3) and the single objective approach using Eq. (1), and three scheduling interval lengths, 100, 200, and 300 simulation time units (STU). These six experiments are replicated 10 times each to facilitate statistical analysis (Pegden et al., 1995).

4. Analysis and results

Readers are reminded that details of results reported here obviously depend on the specific values assigned to parameters that define the simulated job shop. Hence, it is not valid to assume that all conclusions reached in this section are universally applicable to all situations. On the other hand, great care has been taken to select values that are reported in the literature to be representative of real shops and that are consistent with our experience in industry. Further, the analysis that is presented covers a range of conditions and the system performance is reasonably consistent. Because of these facts, we believe it is quite reasonable to expect that the results and conclusions are applicable in a broader sense and submit that they serve as reason to be cautiously optimistic.

Three different analyses are performed on the simulation results. We first examine the average values of the replications at each combination. Then, hypotheses related to the impact that the factors might have on the performance measures are tested using the analysis of variance (ANOVA)

procedure. Finally, the results are subjected to the Student–Newman–Keuls (SNK) test to simultaneously investigate the statistical significance of the mean responses associated with all combinations of the experimental factors. When interpreting the data, remember that all optimization operators are minimized; hence, smaller values of the objective function are better.

4.1. Analysis of means

Averages from the 10 replications of each experiment are presented in Fig. 2 and two interesting observations can be seen. The first confirms the obvious, namely, that employing the multiobjective methodology improves stability at every scheduling interval length. Quantitatively, this improvement in the objective function value is as much as 4%. On the other hand, the multiobjective methodology degrades efficiency for all scheduling lengths. The maximum degradation in average efficiency, however, is less than 1% when compared with the average efficiency using the efficiency-only objective function. This result is smaller than we anticipated and suggests that, for this simulated job shop, the proposed methodology improves stability much more than it degrades efficiency. This result would certainly be particularly attractive in real DJSS situations. The second observation is that lengthening the scheduling interval improves stability and degrades efficiency. This is consistent with intuition since lengthening the rescheduling interval means that fewer jobs are rescheduled so stability is improved. It also means that jobs that arrive soon after a rescheduling point with a due date in the near future must wait a longer time before even having a chance to be scheduled and, hence, degrades efficiency as measured by makespan and tardiness.

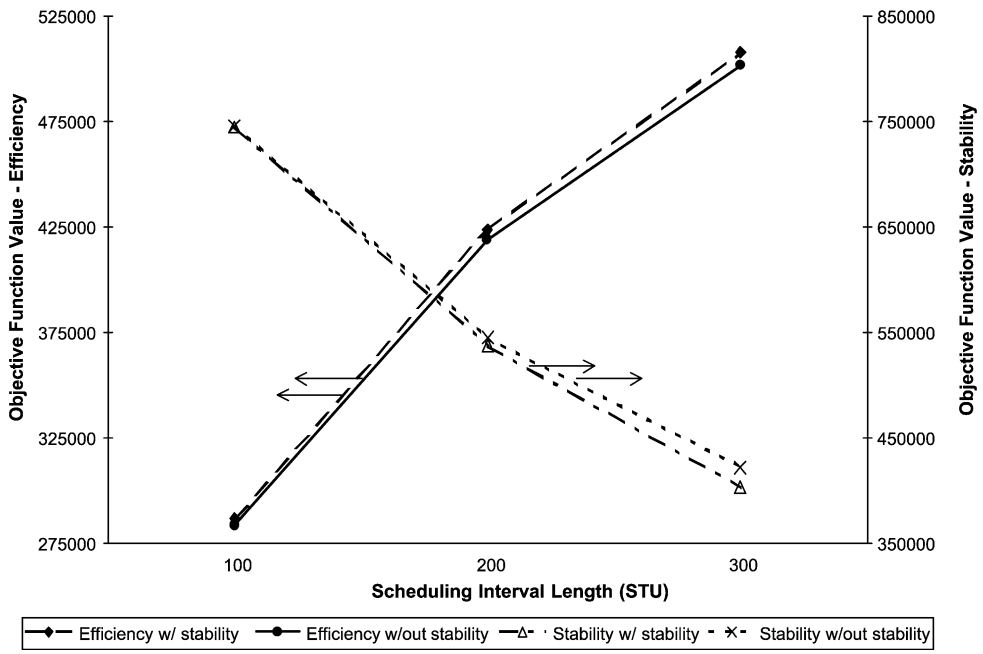

Fig. 2. Results from analysis of means.

4.2. Hypothesis testing and the ANOVA

Table 1 displays the six hypotheses tested in this example. Tests 1 and 4 target the impact, if any, that using the multiobjective methodology has on efficiency and stability, respectively. Tests 2 and 5 address any impact that the different lengths of the scheduling interval have on efficiency and stability, respectively. Finally, tests 3 and 6 seek to determine if the interaction between these two factors has an impact on the two performance measures. The data is analyzed using the General Linear Model (GLM) procedure of SAS (Version 8). The results are presented in Table 1 and, in this study, effects are considered significant if the p value is less than 0.05. Hence, it can be seen that the null hypothesis is rejected in favor of the alternative in tests 2, 4, and 5. This means that the length of the schedule interval has a statistically significant impact on both efficiency and stability and the multiobjective methodology has a statistically significant effect on stability. Tests 3 and 6 have p values in excess of 0.05 so they are judged to be inconclusive and no inference can be drawn from these experiments relative to their hypotheses.

Beyond these statistically accurate statements lies one observation that we find interesting. The data from this example did not support the hypothesis that the multiobjective methodology has a statistically detrimental effect on efficiency. Clearly, this does not mean that it has been proven there is no effect and it is clear that these results might change if the specific parameters of the model are altered; however, from an applied viewpoint this is heartening and certainly deserves further study.

Table 1
ANOVA results

	F value	Pr $> F$
Test 1 H_0: multiobjective methodology does not impact efficiency H_a: multiobjective methodology impacts efficiency	3.84	0.0551
Test 2 H_0: scheduling interval length does not impact efficiency H_a: scheduling interval length impacts efficiency	2857.93	<0.0001
Test 3 H_0: no interaction effect on efficiency H_a: significant interaction effect relative to efficiency	0.14	0.8709
Test 4 H_0: multiobjective methodology does not impact stability H_a: multiobjective methodology impacts stability	4.6	0.0364
Test 5 H_0: scheduling interval length does not impact stability H_a: scheduling interval length impacts stability	2018.84	<0.0001
Test 6 H_0: no interaction effect on stability H_a: significant interaction effect relative to stability	1.47	0.2385

Table 2
Results from Student–Newman–Keuls test

Scheduling interval length (STUs)	Objective function	Efficiency		Stability	
		Mean	SNK grouping	Mean	SNK grouping
100	Without stability	286,781	A	744,869	A
100	With stability	283,691	A	745,939	A
200	Without stability	423,817	B	537,316	B
200	With stability	418,997	B	545,013	B
300	Without stability	507,856	C	403,070	C
300	With stability	501,686	C	422,068	D

4.3. SNK test results

To further investigate the results of tests 2, 4, and 5, the SNK test is performed to determine which of the means belong to statistically similar and different groups. SAS was again used to perform this analysis and the results are summarized in Table 2. In this table, means with the same SNK grouping letter are not significantly different but groups with different letters are considered statistically different. In all cases, the confidence level is assumed to be 5%.

Regarding test 2, the impact of scheduling interval length on efficiency, Table 2 indicates that the efficiency obtained from a scheduling interval length of 100 is statistically different from the efficiency obtained from scheduling interval lengths of 200 and 300 regardless of the type of objective function. Moreover, the efficiency values are the same for both objective functions at each scheduling interval length. Hence, the most efficient schedule is obtained at the scheduling interval length of 100, the least efficient is obtained at scheduling interval length of 300, and it does not matter whether the multiobjective methodology or the single objective function is used to construct the schedule.

Table 2 also suggests that, regardless of type of objective function, the stability measure obtained from scheduling interval length of 100 is statistically different from the stability measure obtained from scheduling interval lengths of 200 and 300. Further, stability measures obtained at scheduling interval lengths of 100 and 200 suggest that there is no statistical difference between the two types of objective functions; however, at a scheduling interval length of 300, the type of objective function does influence the stability obtained (test 4). The schedule has highest stability if it is obtained at scheduling interval length of 300 using the multiobjective methodology and is the least stable at scheduling interval length of 100 with either type of objective function.

5. Conclusions

In this paper, a methodology for dynamic job shop scheduling has been proposed that simultaneously addresses efficiency and stability though a multiobjective approach. The methodology uses periodic rescheduling in which a multicriteria objective function is used as the fitness function of a genetic local search procedure to generate the schedules at each rescheduling point. A simulated job shop is used to investigate the impact of two important aspects of the methodology. To maintain

a practical edge to the analysis, the simulation is built using a structure and parameter values that are documented in the literature as quite typical of real operations and that are consistent with our experience in industry. After validation and verification tests are performed, a full factorial design is used to collect data that is then analyzed in three ways. The results of the experiments and analysis of the example confirm that the using the multicriteria objective function creates more stable schedules; however, it was also found that, for the parameters in this simulation, the efficiency of the more stable schedules was not impacted as severely as intuition might suggest. For the specific numerical example used here, average stability was improved over the range of test conditions by as much as 4% while average efficiency was only degraded by about 1% over the same conditions. ANOVA confirmed that the multicriteria objective function produces a statistically significant improvement in stability but does not produce a statistically significant degradation in efficiency at $\alpha = 0.05$. All tests suggest that the length of the scheduling interval has a significant effect on both efficiency and stability. Now, it is certainly true that these results are dependent on the specific parameters used in the example. On the other hand, we believe that since the simulation was constructed using well-documented information related to real job shops, there is a reasonable chance that these types of results might be found in a variety of industrial environments. At the very least, investigating this observation is an important extension of this research.

Beyond the possibility of producing more stable schedules without adversely affecting efficiency, we believe several other questions and ideas merit further attention. For practitioners, using the experimental methodology to determe the best rescheduling interval for their situation is important. Academically, extending the work in directions suggested by the multiobjective optimization literature might include generating the approximate efficient frontier for the decision maker to select a preferable schedule or developing an interactive methodology to find a desirable efficient solution. A concern that we identified in this paper is the performance measures targeted at stability and how these are quantified. A consistent and defensible approach to this issue would be extremely helpful. Finally, practical problems often have a number of constraints associated with them that are rarely, if ever, considered in this type of research. Adding constraints certainly makes finding good, feasible schedules more difficult but is an important step towards implementation. In conclusion, this research addresses a fundamental problem associated with practical rescheduling, induced instability. A multiobjective approach has been proposed to overcome this problem that shows promise both because it improves stability and because it uses local genetic search to determine a schedule rather quickly. The methodology has been tested using a realistic job shop simulation and the results suggest that stability can be statistically improved with only a minimally adverse impact on classic measures of efficiency.

Appendix A. Genetic local search procedure

A.1. Overview

The genetic local search procedure used in this research is briefly discussed in this appendix. Readers unfamiliar with genetic algorithms might find it beneficial to consult a basic reference text like Goldberg (1989) for background information. The overall approach is as follows:

Step 1 Initialization: An initial population of Z parent schedules is randomly generated.
Step 2 Evaluation: The fitness value is calculated for each parent schedule.
Step 3 Selection: Z pairs of parent schedules are randomly selected.
Step 4 Crossover: A crossover operator is applied to each pair of parent schedules to generate a new schedule.
Step 5 Mutation: A mutation operator is introduced to each offspring.
Step 6 Local Search: A local search procedure is applied to all schedules in the current population.
Step 7 Termination Test: The algorithm is terminated if a stopping condition is satisfied. Otherwise, go back to Step 1.

The fitness function is a simple transformation of Eq. (3) so that it is maximized, a requirement of genetic algorithms.

A.2. Brief discussion of each step

Initialization. An initial population of Z parent schedules is randomly selected. Every parent schedule must be a valid schedule in that (1) it includes all jobs and (2) it does not have any jobs appearing more than once. Since these are randomly built, there is no preference for any job in any location in the schedule.

Evaluation. This step consists of computing the appropriate fitness value of each parent schedule. Start and finish times are computed for each job at each machine after it is verified that schedule is feasible. The fitness value is then computed using Eq. (1) if it is the first schedule generated in a planning horizon or Eq. (3) otherwise.

Selection. To generate an offspring, a pair of parent schedules is required; hence, Z pairs of parent schedules are randomly selected to produce Z offsprings. The random process is biased towards those parent schedules having better fitness values to replicate the 'survival of the fittest' concept. The selection probability of each schedule can be written as shown in the following equation adopted from Ishibuchi and Murata (1998):

$$P(x) = \frac{\mathrm{FV}(x) - \mathrm{FV}_{\min}}{\sum_{x=1}^{Z} \{\mathrm{FV}(x) - \mathrm{FV}_{\min}\}}$$

where

$P(x)$, selection probability of parent schedule x
$\mathrm{FV}(x)$, fitness value of parent schedule x
$\mathrm{FV}_{\min}$, fitness value of the worst schedule.

Crossover. A crossover operator is applied to each pair of parent schedules to generate an offspring. In this study, two-point crossover, illustrated in Fig. A1, is chosen because it outperforms the other crossovers in the literature (Murata et al., 1996a). Two points are randomly selected on one parent schedule. The jobs that are not between these two points are donated to the offspring and the other jobs are placed in the same manner as the other parent schedule.

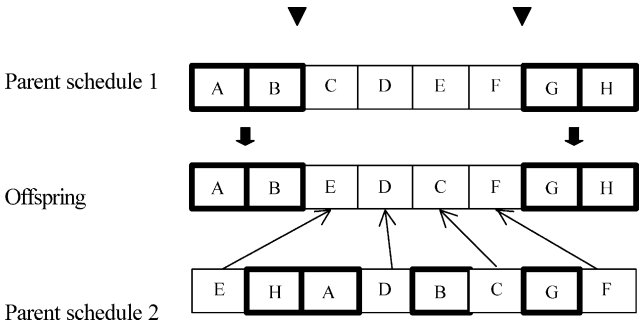

Fig. A1. Two-point crossover (adopted from Murata et al., 1996a).

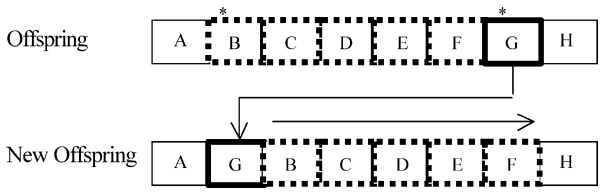

Fig. A2. Shift change mutation (adopted from Murata et al., 1996a).

Mutation. A mutation operator is imposed on each offspring. In this research, a shift change mutation, illustrated in Fig. A2, is chosen because it outperforms the other mutations in the literature (Murata et al., 1996a). Two positions are randomly selected. A job at one position is inserted into another position. The other jobs are shifted to the right of the schedule.

Local search. A local search procedure is applied to each schedule in the population. The procedure employed in the work of Ishibuchi and Murata (1998) is adopted as follows:

Step (0) Acquire an initial schedule x
Step (1) Examine a neighborhood schedule y of the current schedule x
Step (2) If y is a better schedule than x, replace the current schedule x with y and return to Step (1).
Step (3) If randomly chosen r neighborhood schedules of the current schedule x have been already examined, end this procedure. Otherwise, return to Step (1).

References

Baker, K. R. (1984). Sequencing rules and due-date assignment in a job shop. *Management Science*, *30*(9), 1093–1104.
Baker, K. R., & Bertrand, J. W. M. (1981). A comparison of due-date assignment rules. *AIIE Transactions*, *13*, 123–130.
Bertrand, J. W. M. (1983). The effect of workload dependent due-dates on job shop performance. *Management Science*, *29*(7), 799–816.
Bierwirth, C., Kopfer, H. F., Mattfeld, D. C., & Rixen, I. (1995). Genetic algorithm based scheduling in a dynamic manufacturing environment. *Proceedings of the 1995 IEEE International Conference on Evolutionary Computation*, *1*, 439–443.
Blocher, J. D., Chhajed, D., & Leung, M. (1998). Customer order scheduling in a general job shop environment. *Decision Science*, *29*(4), 951–977.

Chang, F.-C. R. (1996). A study of due-date assignment rules with constrained tightness in a dynamic job shop. *Computers and Industrial Engineering*, *31*(1/2), 205–208.

Cheng, T. C. E. (1988). Integration of priority dispatching and due-date assignment in a job shop. *International Journal of Systems Science*, *19*(9), 1813–1825.

Church, L. K., & Uzsoy, R. (1992). Analysis of periodic and event-driven rescheduling policies in dynamic shops. *International Journal of Computer Integrated Manufacturing*, *5*(3), 153–163.

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Reading, MA: Addison-Wesley.

Ishibuchi, H., & Murata, T. (1998). A multi-objective genetic local search algorithm and its application to flowshop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics—Part C: Applications and Reviews*, *28*(3), 392–403.

Kanet, J. J., & Christy, D. P. (1989). Manufacturing systems with forbidden early order departure. *International Journal of Production Research*, *22*(1), 41–50.

Karsiti, M. N., Cruz, J. B., Jr., & Mulligan, J. H., Jr. (1992). Simulation studies of multilevel dynamic job shop scheduling using heuristic dispatching rules. *Journal of Manufacturing Systems*, *11*(5), 346–358.

Lin, N. P., Krajewski, L., Leong, G. K., & Benton, W. C. (1994). The effects of environmental factors on the design of master production scheduling systems. *Journal of Operations Management*, *11*, 367–384.

Morton, T. E., & Pentico, D. W. (1993). *Heuristics scheduling systems*. New York: Wiley.

Murata, T., Ishibuchi, H., & Tanaka, H. (1996a). Genetic algorithms for flowshop scheduling problems. *Computers and Industrial Engineering*, *30*(4), 1061–1071.

Murata, T., Ishibuchi, H., & Tanaka, H. (1996b). Multi-objective genetic algorithm and its applications to flowshop scheduling. *Computers and Industrial Engineering*, *30*(4), 957–968.

Pegden, C. D., Shannon, R. E., & Sadowski, R. P. (1995). *Introduction to simulation using SIMAN*. New York: McGraw-Hill.

Ragatz, G. L., & Mabert, V. A. (1988). An evaluation of order release mechanisms in a job-shop environment. *Decision Sciences*, *19*(1), 167–189.

Rohleder, T. R., & Scudder, G. A. (1993). Comparison of order-release and dispatch rule for the dynamic weighted early/tardy problem. *Production and Operations Management*, *2*(3), 221–238.

Wu, S. D., Storer, R. H., & Chang, P. C. (1993). One-machine rescheduling heuristics with efficiency and stability as criteria. *Computers and Operations Research*, *20*(1), 1–14.

Yamada, T. (1998). Solving the C_{sum} permutation flowshop scheduling problem by genetic local search. *IEEE International Conference on Evolutionary Computation Proceedings: IEEE World Congress on Computational Intelligence, Anchorage, Alaska*, 230–234.